4×4 Matrix Feedback Generator

disting NT plugin — system overview

A disting NT plugin that renders a self-oscillating 4-processor feedback network offline (throttled, non-real-time) into short audio buffers ("kits"), which a real-time sampler engine then plays back as one-shots or loops, sliced and triggered randomly and likely polyphonically. There is no real-time connection between the plugin's input and output — rendering and playback are decoupled by a double-buffered kit store.

1. Signal network

Four channels, one 4×4 feedback matrix mixer, and four fixed injection points (the earlier 4×4 injection mixer has been removed).

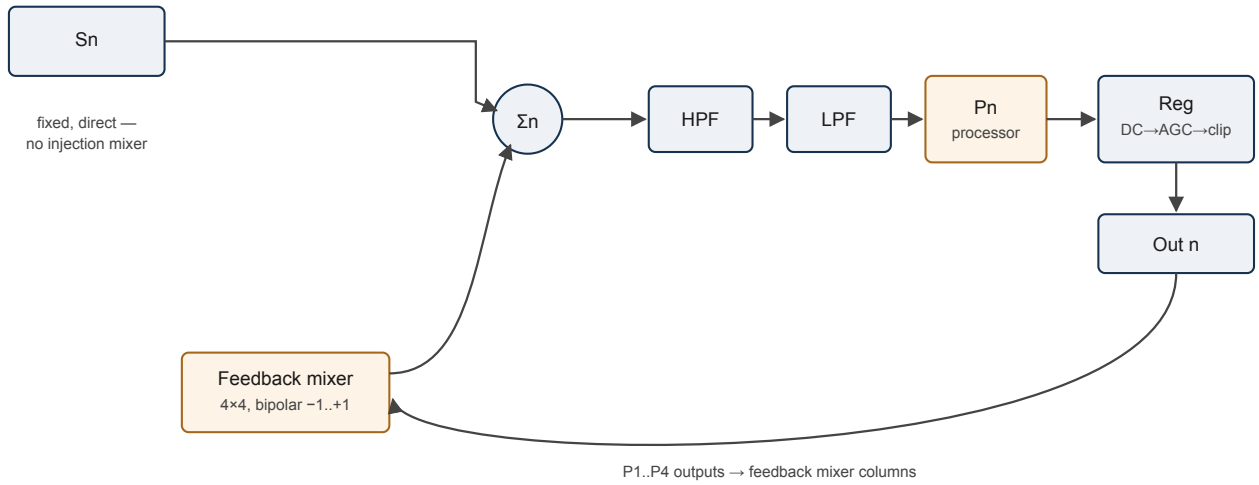


Fig. 1 — Per-channel signal flow. Repeats for n = 1..4; each source feeds only its own channel (fixed, no mixing); the feedback mixer's 16 crosspoints connect every Pn output to every Σn input, covering all 24 elementary loop types (4 self, 6 two-processor, 8 three-processor, 6 four-processor).

- **4 fixed injection points**: each source Sn feeds directly and only into its own channel's Σn. No gain stage, no adjustable level, not a modulation destination. Each Sn's actual content is a math-generated square-wave burst, not live or recorded audio — see "Starter wave injection" below.
- **Feedback mixer** (bipolar attenuverters): every processor output routed to every processor input.
- **Σn**: sums the fixed injection and the feedback mix for channel n.
- **HPF → LPF**: band-limits each channel before its processor; prevents loops from building energy outside the useful range. Ideally CV-controllable.
- **Pn**: four strongly nonlinear processors. **P1** pitch shifter (§2). **P2** frequency shifter (§3). **P3** wavefolder (§4). **P4** bitcrusher/sample-rate-reducer (§5) — all four now defined.
- **Reg** (regulator): DC block → AGC → soft clip (tanh, never hard clip). Sits before the output split, so every loop and every output tap is protected. **AGC target locked 2026-10-18: –18dB to –6dB, relative to the hardware's ±11V absolute output ceiling** — ≈1.39V to ≈5.51V peak. Attack ~2–10 ms, release ~100–500 ms; peak vs. RMS detection still open.
- **Out n**: regulated output, split to the audio output and back into feedback-mixer column n.

Starter wave injection

What actually feeds S1–S4, locked 2026-10-18 after working through several alternatives (live-captured audio, a Contour-generated wavetable) and settling on the simplest: a **math-generated square wave**, not live or recorded audio at all.

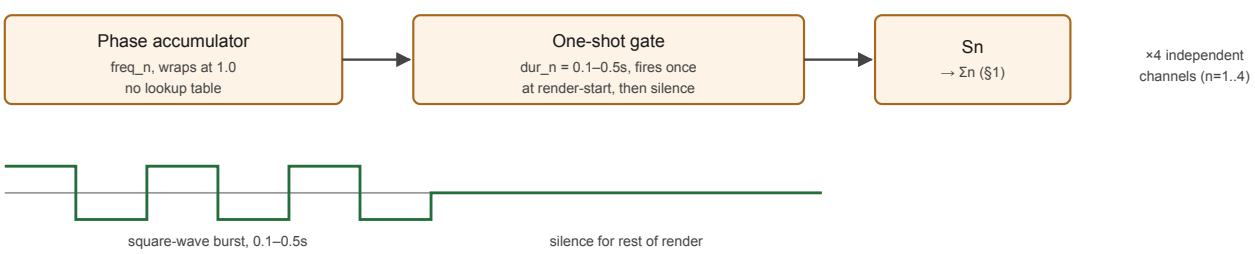


Fig. 7 — Starter wave injection (one of 4 independent channels shown). No live audio capture, no Contour involvement, no memory impact.

- **Algorithm**: a plain phase accumulator per channel compared against a threshold — the cheapest possible waveform generator, no lookup table, no engine state beyond one phase float per channel.
- **4 independent channels**: each of S1–S4 has its own **frequency** and its own **duration**, both real, static `_NT_parameter` entries (performer-set once, not Contour-animated — a one-shot burst this short doesn't have much room for a slow Contour envelope to meaningfully animate within it). 8 new parameters total.
- **Duration**: 0.1–0.5 seconds per channel, independently settable.
- **Timing**: fires once, at the very start of each render's virtual timeline, then silence for the rest of that channel's Sn input. No live audio capture, no recording buffer, no mandatory real-time wait before the throttled render computation begins — this keeps the plugin's original premise intact: **no real-time connection between the plugin's input and output**, since nothing here reads a physical/bus input at all.
- **Memory/CPU impact**: negligible — 4 phase floats + 4 frequency + 4 duration values (≈128 bytes), comfortably within DTC. No Contour instances involved, so this doesn't add to the confirmed 27 Contour-driven destinations or touch the locked kit-buffer memory budget (§8) at all.

2. Pitch-shift processor (P1)

Implements frequency-domain pitch shifting, based on Ocean's algorithm (Juillerat, 2010) — the same technique as the reference implementation this design started from.

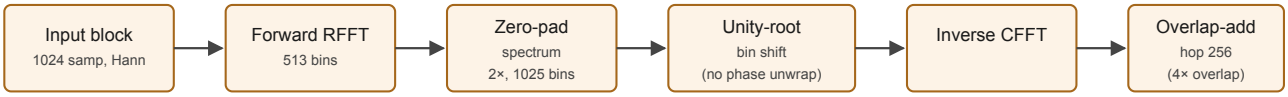


Fig. 2 — Pitch-shift processor pipeline. All blocks are hand-written float C++ (no FFT/DSP library is exposed by the disting NT API), in-place and float32 (not fixed-point), with statically-sized buffers.

The disting NT API exposes **no FFT or DSP library** — the forward/inverse FFT must be hand-written in portable float C++ using only `<cmath>`. This processor's block latency becomes a structural property of whichever feedback loop it sits in — it has no bearing on playback, since playback never has to track the render's timeline in real time.

FFT implementation: in-place, floating-point

In-place: a memory/code-complexity choice only, halving the FFT's own buffer footprint (at the cost of a bit-reversal pass) — matters given the tight static-memory budget (§8–9). **Floating-point, not fixed-point**: audible, not just convenient. Float's rounding error stays proportional to signal level; fixed-point's error is a constant absolute floor that gets relatively louder on quiet material. Since P1 sits inside a feedback loop, any noise it adds recirculates and gets baked permanently into a kit that's then looped/triggered repeatedly. The Cortex-M7's hardware FPU gives no platform reason to prefer fixed-point either.

Parameter

- **Unit**: semitones. **Range**: ±24 (±2 octaves).
- A Contour-animated destination, but unlike the curve-only crosspoint/cutoff/P4 destinations, pitch shift has a **base value** plus a separate **curve-amount (depth)**: the curve offsets around the base rather than replacing it.
- **Base is randomized at each new render, not performer-set — locked 2026-10-18.** Chosen fresh (uniformly within ±24 semitones) at the start of every kit generation, held fixed for that render. No longer a real parameter — internal state. **Depth remains a real, performer-facing parameter.** General rule going forward for any future base+depth-style parameter.
- **Open**: whether this randomization is seed-derived (reproducible) or true hardware entropy (breaks reproducibility for this value) isn't specified yet.
- The driving curve itself is bipolar, **±5V**.

3. Frequency-shift processor (P2)

A second processor implements true frequency shifting using an **IIR allpass-pair Hilbert transformer**, locked 2026-10-06 after comparing it against two other standard approaches:

- **FIR Hilbert transformer** — rejected: expensive per sample, cost scales directly with how low the shift needs to stay accurate.
- **FFT-domain analytic signal** — rejected: comparable per-sample cost to P1 itself, needs its own complex scratch buffer, inherits P1's block-smeared, hop-latency character.
- **IIR allpass-pair (chosen)** — two parallel chains of allpass sections keeping the phase *difference* close to 90° across the audio band, feeding a rotating-phasor complex multiply and a real-part extraction. Recursive, sample-by-sample.

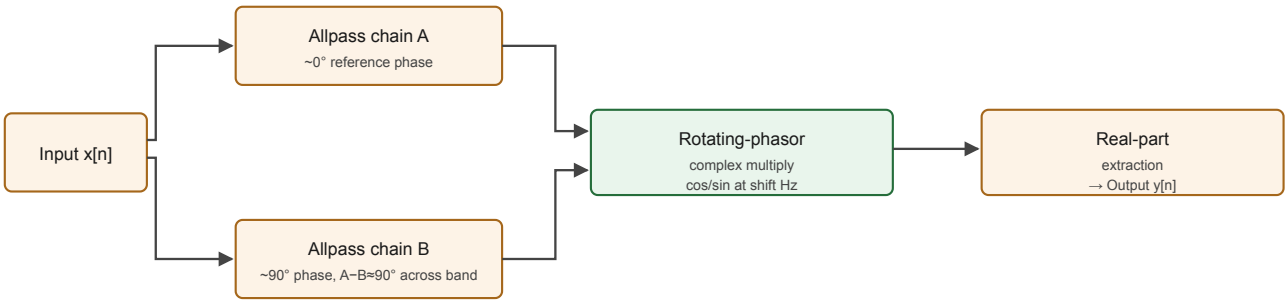


Fig. 3 — P2 frequency shifter: IIR allpass-pair Hilbert transformer + rotating-phasor complex multiply. Recursive and sample-by-sample, chopping cleanly into any render chunk size.

Why this fits: far fewer operations per sample than either alternative (~20–30 vs. ~100+/~80+), naturally sample-by-sample with none of P1's "must accumulate a full hop" bookkeeping, negligible DTC memory, and very short latency instead of P1's block-smeared character.

> **Open gap:** the actual allpass coefficient set needs sourcing from a validated filter-design reference or proper derivation during implementation — not fabricated. Parameter spec (shift unit/range, Contour-destination treatment) is also undefined, unlike P1's.

4. Wavefolder processor (P3)

A third processor implements Buchla/Serge-style wavefolding, with two parameters: **fold** (drive/depth) and **bias** (DC offset, asymmetrizes the fold). Locked 2026-10-06:

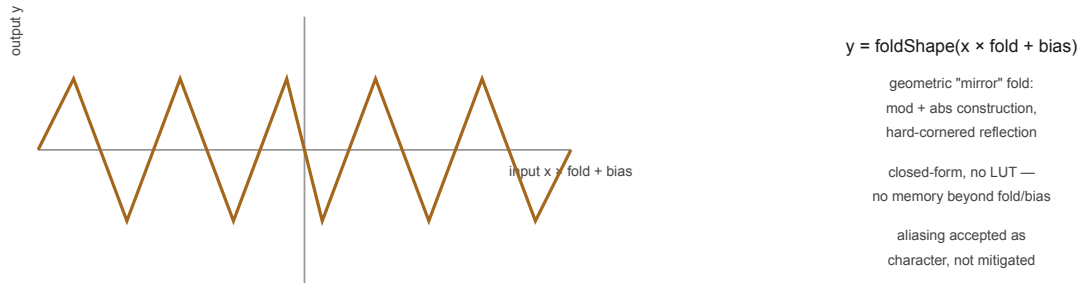


Fig. 4 — P3 wavefolder transfer curve (schematic). Hard-cornered reflections off boundaries rather than a smooth sine-fold.

- **Fold shape: geometric** — harder, more metallic "West Coast" character than a smooth fold, accepting more high-frequency energy at the corners.
- **Evaluation: closed-form**, not a lookup table — DRAM/DTC are already overcommitted (§8); a LUT would add another allocation for a marginal saving.
- **Aliasing: accepted as character, not mitigated.** No oversampling, no ADAA — consistent with how this plugin treats other processors' artifacts (P1's block latency, P2's phase limits, P4's quantization noise).

Bias's steady DC component is removed by Reg's downstream DC blocker; folding maps any input amplitude into a bounded range — a good property inside a self-oscillating loop.

> **Open gap:** the exact geometric fold formula hasn't been derived — only the category of technique is fixed. Parameter spec (unit/range, Contour-destination treatment) is also undefined, same gap as P2's.

5. Bitcrusher / sample-rate-reducer processor (P4)

A fourth processor implements classic digital lo-fi degradation via two independent, curve-only Contour-animated destinations: bit-depth reduction (quantization) and sample-rate reduction (zero-order-hold decimation). Locked 2026-10-18.

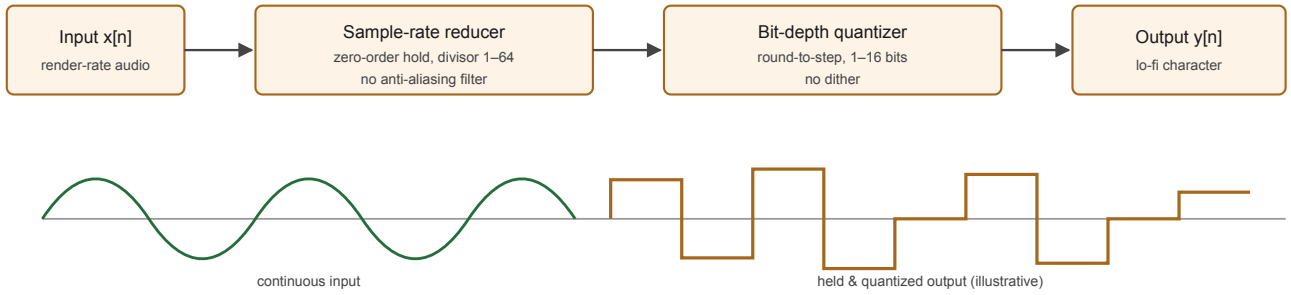


Fig. 5 — P4 bitcrusher/sample-rate-reducer pipeline: zero-order hold followed by round-to-step quantization. Both stages sample-by-sample, no block latency.

Bit-depth reduction (quantizer)

- **Integer-stepped, range 1–16 bits.** 16-bit is effectively transparent; 1-bit is a hard sign-quantizer, degenerating any oscillating input toward a near-square-wave.
- Round-to-nearest-step, no dither: `step = fullScale/(2^bits−1)`, `y = round(x/step)·step`. Quantization distortion is the intended character, consistent with P3's aliasing stance.
- Curve-only Contour destination — no performer-facing base value.

Sample-rate reduction (zero-order-hold decimator)

- **Linear integer divisor, range 1–64**, every integer value reachable. 1 = no reduction; 64 = output updates once every 64 render samples.
- "Max" is the **render's own internal processing rate** (e.g. `NT_globals.sampleRate`, typically 48kHz) — independent of, and upstream of, the already-locked 32kHz kit-storage downsampling (§8), a separate later pipeline stage.
- Classic sample-and-hold, no anti-aliasing filter — the resulting aliasing is the intended lo-fi character, same stance as P3.
- Curve-only Contour destination, same treatment as bit depth.

Why this fits the execution model

Trivial state (one held float + one counter, one current bit-count) — negligible memory alongside P2's allpass state in DTC. Sample-by-sample, no block latency. Self-bounding: quantization stays within half a step of input, held values are literal copies of past samples — neither technique can push the signal beyond the input's own range, same desirable property as P3's folding. Negligible CPU — cheaper than P2.

Both parameters being curve-only adds **2 confirmed Contour-driven destinations** on top of the previously-confirmed 25, bringing the running total to **27** — see §8 for the memory recompute.

6. Execution model: render vs. playback

The plugin's `step()` does two jobs of very different weight.

1. **Player (always-on, real-time):** reads from the *current kit* buffer and plays it back under control of the **Play** parameter (see "Player parameters" below). This is the plugin's entire real-time obligation.
2. **Renderer (background, throttled):** advances the feedback-network computation by one small bounded chunk per `step()` call — an explicit state machine, since `step()` must never block. Produces a fixed-length buffer at whatever pace the chunk size allows.

> **Double buffering is required, not just the simplest option (locked 2026-10-05): playback must be gapless.** A *current kit* and a *next kit* are both kept fully resident; the renderer only writes the inactive one, and the pointer swap happens atomically on completion.
>
> This is close to the only sound approach for the plugin's **eventual** playback model. A kit is rendered as one continuous take and then **sliced and triggered randomly, likely polyphonically** — so there's no single predictable read position a leaner scheme could rely on. Any slice can be hit at any moment, including several overlapping at once, so the entire buffer has to be treated as potentially live for as long as it's current.

Baked parameter-automation curves (pitch shift, feedback amplitude/polarity, HPF/LPF cutoffs, bit depth, sample-rate divisor, etc.) are evaluated against the **renderer's own internal virtual sample-position counter**, not wall-clock time or `step()` call count.

Player parameters (interim), locked 2026-10-18

The sliced/polyphonic player above is the target design (§10), but isn't built yet. For now, playback is deliberately minimal — a stand-in to validate the rest of the signal path before the real playback engine exists:

- **Play**: on/off. Play=1 loops the current kit buffer continuously from start to finish; Play=0 stops — output interpreted as silence (an assumption; confirm if a held-last-value behavior is wanted instead). Real parameter.
- **Output bus**: selects which bus the player writes rendered kit audio to. Real parameter, standard bus-select pattern. **Open**: the API's own convention pairs bus-select with an Output Mode (Add/Replace) parameter via the `_WITH_MODE` macros — whether this plugin needs that too, or just the bare bus-select asked for, isn't decided yet.

The double-buffering requirement above is unaffected by this simplification — an atomic swap into a freshly-rendered kit is needed regardless of how the current one is being played back.

Modulation sources

Motion for the feedback mixer's 16 crosspoint gains, the 8 filter cutoffs, and the processors' own parameters is generated by embedded `contour::Engine` instances (Expert Sleepers' Contour generator — see `CONTOUR_NT_SPEC.md` / `contour_engine.h`) — one instance per destination, not a shared pool.

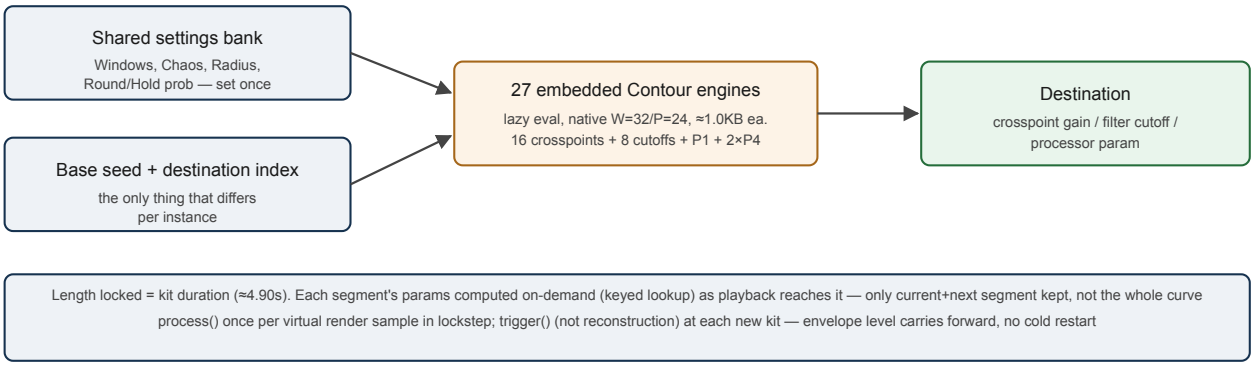


Fig. 6 — Modulation architecture. One shared settings bank drives every instance identically; only the derived seed varies. 27 confirmed destinations after P4 (§5); P2/P3 will add more.

All instances share one global settings bank, set once and applied identically; the only thing that differs per instance is its derived **seed**. **Length is excluded from this bank** — locked to always equal the kit's own maximum duration, derived rather than dialed in.

> **Evaluation is lazy, not baked — committed 2026-10-18 (§8).** Rather than `rebuild()` precomputing and storing the entire render-length breakpoint curve up front, each envelope segment's parameters are computed on demand via Contour's keyed random lookup (`rnd(seed,i,j)`) the moment playback reaches that segment, keeping only the current segment's scalars plus one segment of lookahead (needed because the corner-rounding math blends using the *next* segment's slope). Only the hold/round region `Span` lists still need precomputing, since region placement is a genuine whole-timeline decision. This is a deliberate, explicit deviation from `CONTOUR_NT_SPEC.md`'s "use the reference engine as given," undertaken specifically to free DRAM for kit duration — see §8 for the numbers and the engineering-risk tradeoff.

`process()` must be called exactly once per virtual render sample advanced within a chunk to keep its clock in lockstep with the renderer. At the start of each new kit generation, instances are **re-triggered, not reconstructed**: Contour's envelopes carry their current level forward. `rebuild()` (now scoped to just the region-selection draws rather than every envelope's parameters, likely cheaper than the baked engine's ~700-draw figure — not yet quantified) should be staggered across chunks if many instances need it at once.

Contour's native output is linear volts — maps directly onto crosspoint-gain-type destinations via Level/Offset/Bipolar, but Hz-scaled destinations need an additional mapping layer Contour doesn't provide on its own.

7. CPU budget strategy

- **Always-on cost:** player only — small, constant, every `step()` call.
- **Render cost:** incurred only while actively baking a new kit; chunked so render-chunk cost + player cost + headroom stays well under the CPU-overload threshold on *every single call* — peak per-call cost matters, not average.
- Each Contour instance's `process()` (now also computing the current segment on-demand under lazy evaluation, §6) and occasional `rebuild()` share the same per-chunk budget as the FFT/matrix/filter work.

> Reference point: the LASER example plugin runs 20–30% CPU as a real-time algorithm with a forced per-call cost. This design's forced per-call cost (the player) should be much lower, since the expensive part is optional and schedulable — the actual number can only come from on-device profiling.

8. Memory

All allocation is static, sized via `calculateRequirements()`.

> **Confirmed budget (2026-10-18): DTC 24KB, ITC 64KB, DRAM 512KB** — no separate SRAM allocation is being counted on.

Contour per-instance size: baked vs. lazy

The original reference engine *bakes* — `rebuild()` precomputes and stores the entire render-length curve as breakpoint arrays. Deriving that engine's size from `contour_engine.h`'s array declarations gives, as a function of `kMaxWin` (W) and `kPieces` (P):

`size(W,P) ≈ 1328·W + 160·P + 928` bytes

≈47.3KB at shipped defaults (W=32, P=24), ≈13.2KB at a trimmed W=8/P=12 — both correct, just different trim levels of the same formula.

> **Superseded 2026-10-18: committed to a lazy/on-demand reimplementation instead of the baked engine**, specifically to free DRAM for kit duration. A genuine rewrite of `rebuild()`/`process()`, not a settings change — an explicit, deliberate deviation from `CONTOUR_NT_SPEC.md`'s "use the reference engine as given."
>
> - **Why it's structurally sound**: Contour's randomness is a keyed lookup (`rnd(seed,i,j)`), not a sequential stream — independent draws never disturb each other. Any segment's parameters can be computed the moment playback reaches it, without the whole curve precomputed first.
> - **Still precomputed eagerly**: the hold/round region `Span` lists — a genuine global decision over the whole timeline. The one part that still scales with a trim parameter (`kPieces`), at 32 bytes/piece.
> - **Now lazy**: the envelope/window breakpoint curve. Only the current segment's scalars plus one segment of lookahead (for corner-rounding) are kept, regardless of total window count.
> - **`kMaxWin` no longer needs trimming for memory at all** — footprint is independent of window count. Returns to its native ceiling (32), fully resolving the earlier caveat about capping Windows below its default of 6.
> - **Estimated per-instance size** (design-time estimate, not yet measured): ≈250 bytes fixed + 32·P bytes for region spans. At native `kPieces`=24 (no longer needing a trim either): **≈1,018 bytes ≈ 1.0KB/instance** — a ≈46× reduction from 47.3KB at the same native settings.
> - **Secondary benefit**: `rebuild()` cost likely drops too, since most of the original ~700 draws were for precomputing every envelope up front — now deferred to whenever each segment is reached. Not yet quantified.

Locked kit-buffer / Contour configuration, recomputed at 27 confirmed destinations, lazy Contour

P4 added 2 confirmed curve-only Contour destinations on top of the previously-confirmed 25, bringing the running total to **27**.

Contour approach	Per-instance	×27 instances	Remaining for kit buffer (after ~25KB FFT scratch)	Kit dur./buf., 12-bit packed/32kHz mono
Baked, untrimmed (W=32, P=24)	≈47.3KB	≈1,277KB	**negative — doesn't fit at all**	
Baked, trimmed (W=4, P=8) — prior approach	≈7.34KB	≈198.3KB	≈288.7KB total → ≈144.4KB/buf	≈3.08s
Lazy, native settings (W=32, P=24) — LOCKED	≈1.0KB	≈27.5KB	≈459.5KB total → ≈229.75KB/buf	**≈4.90s**

> **Locked:** lazy Contour evaluation, `kMaxWin`/`kPieces` restored to native values (32/24, no trim needed), kit audio format **12-bit packed, 32kHz, mono**, giving **≈4.90s per kit buffer** at the confirmed 27 destinations — up from ≈3.08s under the prior baked/trimmed approach, with richer untrimmed Contour character as a bonus rather than a tradeoff. Still uses

> essentially the entire DRAM pool once P2/P3 are added, but with ≈459.5KB of margin to absorb them vs. the baked approach's ≈0KB. **Length is locked to always equal the kit's own maximum duration** (≈4.90s, comfortably inside the reference engine's native [1,10]s clamp) — no longer an independent performer-facing field; every render gets exactly one full envelope pass per destination. This also makes Contour's native `Loop` flag moot during rendering.

Item	Approx. size	Notes
Kit buffer, double-buffered — **locked: 12-bit packed, 32kHz, mono, ≈4.90s/buffer**	≈229.75KB/buffer, ≈459.5KB total	Original 5s/mono/float32 spec ≈ 1.9MB double-buffered was ~3.8× the entire DRAM pool, not viable. This replaces it.
FFT/pitch-shifter scratch (in-place, 2048-sample zero-padded complex buffer)	~20–25KB	Too large for 24KB DTC — must live in DRAM.
Matrix coefficients, filter/regulator state (P2 allpass, P3 fold/bias, P4 hold+quantizer)	negligible, ~1KB	Comfortable fit in DTC.
Contour generators — **locked: lazy, native W=32/P=24**	≈1.0KB/instance × 27 confirmed destinations ≈ 27.5KB	Estimate pending implementation/measurement (§10). Destination count will still grow once P2/P3 are defined, but with margin to absorb it.
Baked automation curve tables	n/a	Resolved in favor of inline, lazy `process()` evaluation — avoids duplicating kit-buffer-scale memory, and now avoids duplicating per-destination curve storage too.

9. Parameters vs. internal state

Only values the performer needs to control — via pot, encoder, CV, MIDI, or I2C, visible on a GUI parameter page — need to be real `_NT_parameter` entries routed through `self->v[]`. A plugin's parameter array is fixed-size and finite, reserved for what's actually useful to a performer.

Everything else is ordinary instance-struct data, with no involvement from the parameter system:

- The feedback mixer's 16 crosspoint gains
- The 8 filter cutoffs
- P1's randomized base pitch-shift value (fresh each render, not performer-set)
- P2's allpass filter states (coefficients TBD)
- P3's fold/bias pre-scale values (exact formula TBD)
- P4's current bit-depth and sample-rate-divisor values (curve-only, no base parameter)
- Each Contour instance's generated curve state and its derived seed
- The renderer's virtual sample-position counter
- Kit-buffer pointers and double-buffer swap state

> **If any of this internal state must survive a preset save/reload** — e.g. a baked curve authored per-preset rather than fixed at compile time — the correct mechanism is `serialise`/`deserialise` (`serialisation.h`), fully decoupled from the parameter system.

What *does* likely need to be a real parameter — triggering a new render, selecting a kit slot, a macro like "intensity" — is not yet finalized. **Play and Output bus are now confirmed, locked real parameters (§6)** — a separate overall output level control remains unresolved. P1's depth is a real parameter; P1's base is not — randomized each render, internal state. P4's bit-depth and rate-divisor are curve-only internal state, not parameters. Whatever P2/P3 settle on will be real parameters only for their depth-equivalent, if they use base+depth treatment like P1's — their base, if any, would be randomized the same way, never performer-set.

> **One exception worth flagging:** the Contour engines' shared settings bank (Windows, Chaos, Radius, Round/Hold prob, etc. — Length is excluded, locked to the kit's own duration rather than performer-set) reads naturally as a real, performer-facing "character" page rather than internal state. Whether it ends up as real parameters or compiled-in constants isn't finalized, but it's the one part of this subsystem that plausibly belongs on a parameter page.

> **Confirmed real parameters so far, locked 2026-10-18 (11 total):**
>
> | Parameter | Count | Notes |
> |---|---|---|
> | Starter frequency (per channel, S1–S4) | 4 | Static, not Contour-animated (§1) |
> | Starter duration (per channel, S1–S4) | 4 | 0.1–0.5s, static (§1) |
> | P1 pitch-shift depth | 1 | Base is randomized internal state, not a parameter (§2) |
> | Play | 1 | Loops the current kit on/off (§6) |
> | Output bus | 1 | Selects the player's output bus; Output Mode pairing still open (§6) |
>
> None of these are Contour-animated or internal state — all plain, static, performer-facing values.

10. Open design questions

Nothing blocks starting to code the renderer scaffolding, the matrix/feedback plumbing, P1, P4, or the Contour integration — all four processor slots are now defined and the memory budget has a locked, concrete operating point. P2 and P3 remain individually blocked on their own open gaps below — those block writing those two processors specifically, not the rest of the plugin.

Resolved this session

- ~~P4 was completely undefined~~ — resolved: bitcrusher/sample-rate-reducer, both parameters curve-only Contour destinations, locked 2026-10-18 (§5).
- ~~Contour per-instance memory size conflict~~ — resolved: different trim levels of the same formula, not a real contradiction (§8).
- ~~The "~40 destinations" figure~~ — resolved: confirmed baseline is now 27, pending P2/P3.
- ~~SRAM pool size~~ — DTC 24KB / ITC 64KB / DRAM 512KB is the complete budget; no separate SRAM allocation counted on.
- ~~Kit buffer bit-depth / sample-rate / duration / Contour trim level~~ — **locked**: 12-bit packed/32kHz mono, ≈4.90s/buffer at 27 destinations (recomputed after both P4 and the lazy-Contour switch). Still uses essentially the entire DRAM pool once P2/P3 are added, but with ≈459.5KB of margin vs. the baked approach's ≈0KB.
- ~~What actually feeds S1–S4~~ — resolved: a math-generated square wave per channel (phase accumulator), 4 independent frequencies + 4 independent durations (0.1–0.5s), all static real parameters, fired once at render-start. No live audio capture, no Contour involvement, no memory impact — the plugin's original "no real-time connection between input and output" premise holds intact (§1).
- ~~Reg's AGC target level~~ — resolved: –18dB to –6dB relative to the hardware's ±11V output ceiling (≈1.39V–5.51V peak), locked 2026-10-18 (§1). Peak vs. RMS detection still open.
- ~~Contour memory vs. kit duration~~ — resolved: committed to a lazy/on-demand Contour reimplementation instead of the baked engine (§6/§8), cutting per-instance size from ≈7.34KB (trimmed) to an estimated ≈1.0KB (native settings) — a ≈46× reduction. Also fully resolves the former Windows/kMaxWin=4 caveat: the lazy design needs no window-count trim at all.
- ~~P1 base value treatment~~ — resolved: randomized fresh at each render, not performer-set; no longer a real parameter (§2). Depth remains a real parameter. General rule for any future base+depth-style parameter too.
- ~~Playback model for now~~ — resolved (interim): a single **Play** parameter loops the whole kit buffer on/off, replacing the eventual sliced/polyphonic design for early testing purposes only (§6). The eventual design is unchanged, just not built yet.
- ~~Output routing~~ — resolved: a real **Output bus** parameter selects where the player writes rendered kit audio (§6) — closes the gap flagged in the previous session.

New from this session, not blocking the rest of the plugin

- **Lazy Contour engine needs actual implementation and measurement.** The ≈1.0KB/instance figure (§8) is a design-time estimate, not a measured `sizeof()` — the engine doesn't exist yet. The one-segment-lookahead handling for corner-rounding is the trickiest part; needs testing against the reference engine's golden test values (`CONTOUR_NT_SPEC.md` §8) to confirm equivalent output.
- **Is randomized-base generation seed-derived or true entropy?** P1's base (§2), and any future base+depth parameter's base, needs randomizing at each render — but whether that draw is seed-keyed (reproducible) or hardware entropy (breaks reproducibility for just this value) isn't specified.
- **Does Output bus need a paired Output Mode parameter?** The API's convention pairs bus-select with Add/Replace via the `_WITH_MODE` macros, as used for every other audio output in this project's reference examples (§6) — whether this plugin follows that or just uses the bare bus-select asked for isn't decided.
- **Play=0 behavior**: interpreted as silent output (an assumption, §6) — confirm this is intended rather than e.g. holding the last played value.

Not blocking the rest of the plugin, but unresolved

- **P2 allpass coefficients and parameter spec:** approach locked, coefficients and parameter treatment not yet defined. Blocks writing P2 specifically.
- **P3 exact fold formula and parameter spec:** category locked, precise formula and parameter treatment not yet defined. Blocks writing P3 specifically.
- **State continuity between renders:** resolved for Contour-driven destinations — open for processor-internal filter memory, AGC envelope state, and the feedback loop's own settling.
- **Render chunk granularity and bake time:** only determinable by on-device profiling.
- **Sampler slicing method:** model settled (continuous render, random/polyphonic trigger) — slice count, method, polyphony limit unspecified. The current interim Play-loop player (§6) is a placeholder standing in for this until it's built.
- **AGC vs. pitch-shifter latency interaction:** possible periodic pumping tied to hop boundaries.
- **Kit-swap continuity:** double buffering guarantees no torn reads, not a smooth musical transition at the swap instant.
- **Output-unit mapping layer:** linear volts → Hz-type destinations not yet designed.
- **Contour settings: parameter or constant?** Reads naturally as a parameter page, not explicitly finalized.

11. Platform constraints honored

- No dynamic allocation — placement `new` on firmware-provided static memory only.
- No exceptions, no RTTI, no STL, no threads, no OS/file calls inside `step()`.
- FFT implemented from scratch in portable float C++ (`<cmath>` only) since the API provides none.
- `step()` never blocks — render work is chunked as an explicit state machine, never a single long pass.
- Contour's reference engine already meets these constraints as shipped (no heap, double-precision via the Cortex-M7 hardware FPU, placement `new` only) — reused as-is, not re-derived.